Guion Personal de Jose - Sprint 1 HELIOS

> *Documento personal de Jose Brown (Product Owner) para la presentacion del viernes. Cubre slides 5, 6, 7 y 8 con explicaciones en lenguaje simple. Duracion de tu parte: 65 segundos.*

INDICE

1. Tu guion exacto (que decir en cada slide)
2. Explicacion de cada termino - Slide 5 (ISTQB)
3. Explicacion de cada termino - Slide 6 (TMMi + TQM)
4. Explicacion de cada termino - Slide 7 (Metricas, Herramientas, Kanban)
5. Como demostrar cada herramienta si la profesora pide ver
6. Como explicar el uso de IA asistente (los tests)
7. Preguntas anticipadas con respuestas en lenguaje simple
8. Tips de presentacion
9. Resumen ultra-rapido

SECCION 1: TU GUION EXACTO

Tiempo total: 65 segundos (slides 5, 6, 7 y cierre 8).

SLIDE 5 - "Lo que medimos segun ISTQB" (20 segundos)

> *"Como Product Owner les muestro lo que medimos segun el estandar ISTQB. Cuatro indicadores clave:*
>
> ***55 pruebas automatizadas** que pasan en cada deploy, hechas con Vitest. Tenemos un umbral minimo del 60% activado: si baja, el build falla.*
>
> ***61.4% de cobertura de codigo** en sentencias, y **90.9% en funciones**.*
>
> ***300 milisegundos** de latencia P95 con 200 usuarios concurrentes usando Locust. La meta era 2 segundos: la superamos 6 veces.*
>
> *Y **17 defectos cazados y resueltos**, con una tasa de **7.4 bugs por mil lineas de codigo**, todos trazables en el repositorio."*

SLIDE 6 - "Calidad del proceso y mejora continua" (20 segundos)

> *"En **TMMi**, que es el modelo de madurez de pruebas, estamos en **nivel 2 - Gestionado, camino a nivel 3**. Cumplimos: politica de pruebas documentada, plan trazable (cada bug tiene su test), metricas registradas cada sprint, code review obligatorio antes de hacer merge, y threshold automatico en CI.*
>
> *En **TQM**, que es mejora continua, lo demostramos asi: yo como Product Owner pruebo el deploy real antes de cada demo, los bugs detectados en vivo se arreglan en la misma sesion, cada bug genera su test de regresion, y reconocemos honestamente las deudas tecnicas."*

SLIDE 7 - "Metricas + Herramientas + Kanban" (20 segundos)

> *"Aqui esta el registro completo categorizado:*
>
> ***Requerimientos:** 5 historias de usuario completadas al 100%. **Diseno:** 9 componentes, 6 tablas SQL, 1 funcion atomica. **Codigo:** 2.3 mil lineas, con 600 lineas dedicadas a tests, y 9*

> *commits firmados.* ***Esfuerzo:*** *53 puntos de historia, 37 completados, 5 en progreso.*
> ***Productividad:*** *0.28 mil lineas por persona, con 2 deploys al dia.*
>
> *Usamos 6 herramientas: GitHub, Jira, Vitest, Locust, Supabase y Railway.*
>
> *El tablero Kanban en Jira tiene 17 bugs y las 5 historias del Sprint 1 en Done, cancelar y reprogramar cita en progreso, y 5 historias listas para Sprint 2."*

SLIDE 8 - Cierre (5 segundos)

> *"MVP funcional, metricas medidas, proceso documentado. Gracias. Preguntas?"*

SECCION 2: EXPLICACION DE CADA TERMINO - SLIDE 5 (ISTQB)

1. Tests automatizados

Que es: Son pequenos programas que verifican automaticamente que el codigo funcione bien. Cada vez que cambias algo, los tests corren solos y te avisan si algo se rompio.

Analogia: Es como una fabrica de autos. Antes de que un carro salga, pasa por 55 estaciones que verifican luces, frenos, motor, etc. Si alguno falla, el carro NO sale. Igual con el codigo: si un test falla, el deploy NO sucede.

Como funciona tecnicamente:

1. Programador hace cambio en el codigo
2. Antes de subirlo, corre `pnpm vitest run`
3. Las 55 pruebas se ejecutan en 12 segundos
4. Si TODAS pasan, se sube el codigo
5. Si UNA falla, se corrige antes de subir

Si pregunta que prueban: "Tres cosas: que los componentes visuales se rendericen bien, que las validaciones funcionen (ejemplo: cedula 8-123-4567), y que los flujos criticos manejen tanto el camino feliz como los errores."

2. Cobertura de codigo (61.4%)

Que es: Que porcentaje de tu codigo esta siendo probado por los tests automatizados.

Analogia: Imagina que tu casa tiene 100 habitaciones y un guardia revisa 61 cada noche. Las otras 39 no se revisan. Cobertura = 61%.

En codigo: tenemos 2,300 lineas. Los tests prueban el 61% de ellas (las criticas). El 39% restante es codigo de estilos visuales y configuracion que no tiene logica que se pueda romper.

Por que NO es 100%: Cubrir el 100% es imposible y absurdo. El estandar de la industria es 60-80%. Mas de eso es desperdicio.

Si pregunta por que no 100%: "Porque cubrir el 100% es ineficiente. Enfocamos las pruebas en la logica critica: autenticacion, agendamiento, validaciones. Los archivos de estilos visuales no necesitan tests porque no tienen logica que falle."

3. Latencia P95 = 300 ms

Que es: Cuanto tarda la app en responder cuando hay muchos usuarios al mismo tiempo.

- **Latencia** = tiempo de respuesta (en milisegundos)
- **P95** = "Percentil 95" = el 95% de las peticiones responden en menos de ese tiempo

Analogia: Imagina una tienda con 200 clientes al mismo tiempo. El cajero atiende:

- 190 personas en menos de 300 milisegundos (P95 = 300 ms)
- Las otras 10 (las mas lentas) pueden tardar un poco mas

Por que P95 y no promedio: un promedio puede ocultar que 5 personas esperaron 10 segundos. P95 te dice "el 95% recibe servicio en maximo X tiempo". Es honesto.

Comparalo con la meta:

- Meta del curso: menos de 2 segundos (2000 ms)
- Nosotros logramos: 300 ms
- Es 6.6 veces mas rapido que la meta

Si pregunta como lo midieron: "Con Locust, una herramienta open-source de pruebas de carga. Escribimos un script que simula 60% consultando horarios, 30% agendando citas y 10% revisando sus citas. Lo corrimos con 200 usuarios virtuales al mismo tiempo."

4. 17 defectos cazados (tasa 7.4 bugs/KLOC)

Que es: 17 errores encontrados y arreglados durante el sprint. Cada uno tiene un ID (B-01 a B-17), su causa documentada y su solucion registrada en GitHub.

KLOC = "Kilo Lines Of Code" = mil lineas de codigo. **7.4 bugs/KLOC** = encontramos 7.4 errores por cada mil lineas de codigo.

Analogia: Es como decir "encontramos 7 erratas por cada mil palabras escritas". En la industria:

- Menos de 5 bugs/KLOC = software muy maduro (Google, Microsoft)

- 5-10 bugs/KLOC = software profesional normal (AQUI ESTAMOS)
- Mas de 15 bugs/KLOC = codigo de baja calidad

Por que es BUENO tener bugs documentados: Si dijeramos "0 bugs" seria mentira. Encontrar y documentar bugs demuestra disciplina profesional. El bug que no detectas hoy explota manana en produccion.

Si pregunta si no es mucho: "Al contrario, es honestidad profesional. Cada bug fue detectado por nosotros antes del demo, arreglado en la misma jornada y le agregamos un test de regresion para que no vuelva a pasar. Si hubieramos escondido los bugs, la metrica seria 0 pero el sistema fallaria en produccion."

SECCION 3: EXPLICACION DE CADA TERMINO - SLIDE 6 (TMMi + TQM)

TMMi - Madurez del proceso (5 puntos)

Que es TMMi

Modelo internacional del 1 al 5 que mide que tan organizada esta una empresa para hacer pruebas:

- **Nivel 1:** Caos. Nadie sabe que hace.
- **Nivel 2:** Gestionado. Tenemos proceso definido (AQUI ESTAMOS)
- **Nivel 3:** Definido. Procesos estandarizados.
- **Niveles 4-5:** Optimizado con metricas avanzadas (empresas tipo Google).

Los 5 puntos de la slide

1. Politica de pruebas documentada

Tenemos un documento escrito que dice "asi se hacen las pruebas en este equipo". Esta en el archivo CLAUDE.md del repositorio.

> *Frase: "Tenemos una politica de pruebas escrita en el archivo CLAUDE.md. Define como se hacen las pruebas, que cobertura minima exigimos, que herramientas usamos."*

2. Plan de pruebas trazable (cada bug tiene su test)

Cuando aparece un bug, no solo lo arreglamos: escribimos un test que verifica que el bug esta arreglado. Ese test corre para siempre.

> *Frase: "Cada bug que detectamos genero un test de regresion especifico. Por ejemplo, el bug B-10 del spinner eterno tiene 3 tests que verifican que el boton se rehabilite tras un timeout."*

3. Metricas registradas por sprint

Cada sprint anotamos: cuantos bugs, cuanto codigo, cuanta cobertura, cuanta latencia. Es historico.

> *Frase: "Registramos las metricas en cada sprint. Asi podemos comparar evolucion: cuantos bugs en Sprint 0, cuantos en Sprint 1, tendencias."*

4. Code review obligatorio antes de merge

ANTES de que cualquier codigo entre al sistema principal, otra persona DEBE revisarlo. Nadie sube codigo solo.

Usamos doble capa: - Agente revisor de IA que revisa el codigo primero - Equipo revisa tambien - Ninguna linea llega a produccion sin pasar por revision

> *Frase: "Tenemos code review obligatorio antes de cada merge a produccion. Usamos asistencia de IA para una primera pasada y el equipo valida despues. Ninguna linea de codigo llega a produccion sin pasar por revision."*

5. Threshold automatico en CI

- **CI** = "Continuous Integration" = sistema automatico que se activa cada vez que alguien sube codigo
- **Threshold** = umbral minimo de calidad

Significado completo: Tenemos un guardia automatico que dice: "Si la cobertura baja del 60%, NO dejo pasar este codigo a produccion". Es una regla en el archivo `vite.config.ts`.

Analogia: Es como un semaforo automatico. Si el codigo no cumple el minimo (verde), no avanza. Si lo cumple, avanza. No depende de que un humano se acuerde - el sistema lo hace solo.

> *Frase: "Threshold automatico en CI significa que el sistema tiene una regla automatica: si la cobertura de tests baja del 60%, el deploy se rechaza solo. No depende de que alguien se acuerde de revisar."*

TQM - Mejora continua (5 puntos)

Que es TQM

Total Quality Management = filosofia donde la calidad no es una fase final, sino que esta en TODO el proceso. Cada error es una oportunidad para mejorar.

Los 5 puntos

1. PO prueba el deploy real antes de cada demo

Tu haces esto. Antes de mostrar al cliente, tu entras al sistema en produccion y verificas que todo funcione.

> *Frase: "Yo, como Product Owner, pruebo personalmente el deploy real antes de cada demo. No confiamos solo en los tests automaticos. Hoy mismo detecte y reportamos un bug que arreglamos en 2 horas."*

2. Bugs detectados en vivo se arreglan en sesion

Cuando tu detectas un bug, lo arreglamos el mismo dia, no en 3 semanas.

> *Frase: "Cuando detectamos bugs en uso real, los arreglamos en la misma sesion de trabajo. Por ejemplo, hoy detectamos el bug del registro y lo resolvimos en menos de 3 horas, con tests + deploy."*

3. Cada bug genera test de regresion

Cada bug arreglado tiene un test que verifica que nunca vuelva a pasar.

> *Frase: "Cada bug que se arregla genera un test de regresion nuevo. Si en el futuro alguien rompe ese codigo por accidente, el test fallara y nos avisara. Por eso tenemos 55 tests."*

4. Plan Mode obligatorio antes de tocar codigo

Antes de cambiar cualquier codigo, escribimos un plan completo y lo aprobamos. Nadie improvisa.

> *Frase: "Antes de hacer cualquier cambio de codigo, escribimos un plan completo en Plan Mode. El plan se aprueba antes de empezar a programar. Esto evita rehacer trabajo."*

5. Documentacion viva (CLAUDE.md, decisiones)

La documentacion NO es papel muerto. Se actualiza con cada decision tecnica. Vive con el codigo.

> *Frase: "Tenemos documentacion viva en CLAUDE.md. Cada decision tecnica importante queda anotada con su razon. No es documentacion que se hace al final y se olvida."*

6. Honestidad: deudas tecnicas reconocidas

Reconocemos abiertamente que FALTA o esta a medio terminar.

> *Frase: "Reconocemos honestamente las deudas tecnicas: nos falta email de confirmacion, MFA, monitoreo profesional. Esta documentado en el roadmap del Sprint 2. No escondemos las limitaciones."*

SECCION 4: EXPLICACION DE CADA TERMINO - SLIDE 7 (Metricas + Herramientas + Kanban)

PARTE 1 - Metricas categorizadas (las 6 filas)

REQUERIMIENTOS - 5 US (US-01 a US-05), 100% completadas

"User Stories" = lo que pidio el cliente (la profesora en este caso). Son 5 funcionalidades. Las 5 estan terminadas.

> *Frase: "5 historias de usuario en el Sprint 1, completadas al 100%. Desde seleccionar especialidad hasta consultar mis citas."*

DISENO - 9 componentes, 6 tablas SQL, 1 RPC atomica

- **9 componentes** = 9 piezas reutilizables de la interfaz (botones, formularios, calendario). Pensados para usar en varias partes.
- **6 tablas SQL** = 6 estructuras de datos en la base de datos: sedes, especialidades, medicos, horarios, pacientes, citas.
- **1 RPC atomica** = una funcion especial en la base de datos llamada `crear_cita_atomica`. Garantiza que dos pacientes no puedan reservar el mismo horario al mismo tiempo.

Analogia RPC atomica: Es como cuando compras boletos online y el sistema "reserva" el asiento por 5 minutos. Nadie mas puede comprarlo en ese tiempo.

> *Frase: "9 componentes reutilizables del frontend, 6 tablas en la base de datos, y 1 funcion SQL atomica que evita reservas duplicadas del mismo horario."*

CODIGO - 2.3 KLOC, 600 LOC tests, 9 commits firmados

- **2.3 KLOC** = 2,300 lineas de codigo en total
- **600 LOC tests** = 600 lineas dedicadas SOLO a tests (el 26% del codigo son pruebas)
- **9 commits firmados** = 9 cambios subidos a GitHub con identidad verificada (cada miembro firmo al menos uno)

Dato impresionante: El 26% de nuestro codigo son TESTS. Eso es muy bueno. En la industria se considera saludable mas del 20%.

> *Frase: "2.3 mil lineas de codigo en total, de las cuales 600 son dedicadas a tests. Eso es un 26% del codigo. Y 9 commits firmados, minimo uno por integrante del equipo."*

DEFECTOS - 17 bugs cazados, 17 resueltos, 7.4 bugs/KLOC

Ya explicado en Slide 5. 17 bugs encontrados, 17 arreglados, ninguno pendiente.

> *Frase: "17 bugs detectados, 17 resueltos. Cero pendientes. Tasa de 7.4 por mil lineas, en rango profesional."*

ESFUERZO - 53 SP, 37 completados, 5 en progreso

- **SP** = "Story Points" = puntos de esfuerzo (1 SP = aprox medio dia de trabajo)
- **53 SP** = total de trabajo planeado
- **37 completados** = ya hechos
- **5 en progreso** = en proceso

Calculo: 37/53 = 70% del sprint completado.

> *Frase: "Estimamos 53 puntos de esfuerzo para el Sprint 1. Completamos 37, hay 5 en progreso. Eso es el 70% del Sprint terminado. El resto pasa al Sprint 2."*

PRODUCTIVIDAD - 0.28 KLOC/persona, 2 deploys/dia

- **0.28 KLOC/persona** = cada miembro escribio en promedio 280 lineas de codigo
- **2 deploys/dia** = en promedio sacamos 2 versiones nuevas a produccion cada dia

Por que es bueno: En la industria, hacer 2 deploys/dia se considera ritmo profesional agil. Empresas como Amazon hacen miles. Pero para equipos pequenos, 2 al dia es excelente.

> *Frase: "En promedio 280 lineas de codigo por persona y 2 despliegues a produccion por dia. Es ritmo agil profesional."*

PARTE 2 - Las 6 Herramientas

1. GitHub - 9 commits

Plataforma donde vive el codigo. Cada cambio se "comitea" con la firma de quien lo hizo.

> *Frase: "GitHub guarda todo el codigo con historial de cambios. 9 commits firmados."*

2. Jira - 21 issues

Plataforma de gestion de tareas. Cada tarea es un "issue".

> *Frase: "Jira es nuestro tablero Kanban con 21 issues organizados."*

3. Vitest - 55+ tests (CLI, sin plataforma web)

Programa que ejecuta los 55 tests que nosotros escribimos. No es plataforma web, es herramienta de linea de comandos.

> *Frase: "Vitest es el framework que ejecuta los 55 tests automatizados. Se corre con un comando en el terminal: pnpm vitest run. Es el estandar moderno para proyectos con Vite y TypeScript."*

4. Locust - P95 300ms (CLI, sin plataforma web)

Programa Python que simula muchos usuarios al mismo tiempo para medir cuanto aguanta tu app.

Analogia: Es como hacer un "simulacro de evento" en un cine. Antes de la premier real, metes 200 personas para ver si las taquillas, banos y filas aguantan. Locust hace eso con tu app.

> *Frase: "Locust es la herramienta que usamos para pruebas de carga. Simula 200 usuarios concurrentes haciendo peticiones reales. Con eso medimos la latencia P95 de 300 milisegundos."*

5. Supabase - BD + Auth

Servicio cloud que da 3 cosas en una:

1. **Base de datos PostgreSQL** (BD)
2. **Sistema de autenticacion de usuarios** (Auth)
3. **API REST automatica** para hablar con la BD

Por que es bueno: Sin Supabase, tendriamos que programar nosotros la BD + el login + la API. Eso es semanas de trabajo. Supabase lo da hecho.

> *Frase: "Supabase nos da base de datos PostgreSQL, autenticacion de usuarios y API REST en un solo servicio. Nos ahorro semanas de desarrollo backend."*

6. Railway - Deploy CI

Plataforma donde esta alojada nuestra app en internet. Cuando subimos codigo a GitHub, Railway lo despliega automaticamente.

Analogia: Railway es el local donde esta montada la tienda. GitHub es el almacen donde guardamos los productos. Cuando agregamos productos nuevos al almacen, Railway los pone en la vitrina automaticamente.

Deploy CI: "CI" = Continuous Integration = despliegue automatico continuo. Cada vez que pasa los tests, sale a produccion.

> *Frase: "Railway es donde esta alojada la app en produccion. Cuando hacemos push a GitHub, Railway despliega automaticamente. La URL es helios-web-production.up.railway.app."*

PARTE 3 - Tablero Kanban Jira

Las 3 columnas

Done (Hecho): 17 bugs + 5 US Sprint 1. Lo que ya terminamos.

In Progress (En progreso): Cancelar/reprogramar cita. Lo que estamos haciendo ahora (es del Sprint 2).

To Do (Por hacer): 5 historias para Sprint 2. Lo que viene: confirmaciones por email, notificaciones, etc.

> *Frase: "El tablero Kanban en Jira refleja el estado real: 17 bugs y 5 historias del Sprint 1 en Done. Cancelar y reprogramar cita en progreso. 5 historias pendientes para Sprint 2."*

SECCION 5: COMO DEMOSTRAR CADA HERRAMIENTA

Si la profesora dice "muestrame X plataforma":

Herramienta	Como demostrar	URL o accion
GitHub	Abrir el repo en navegador	github.com/MarianC-programing/Helios
Jira	Abrir el tablero Kanban	helios-group.atlassian.net
Vitest	Abrir terminal y correr	pnpm vitest run (muestra 65/65 OK)
Locust	Mostrar locustfile.py en GitHub	scripts/locustfile.py
Supabase	Dashboard de Supabase	supabase.com/dashboard/project/uobhbikcctausgpnzsph
Railway	Dashboard de Railway	railway.app, proyecto helios-citas

Como demostrar Vitest en vivo (lo mas complicado)

Opcion A - Si tienes tu laptop a la mano:

1. Abrir PowerShell o CMD en tu PC
2. Ir a la carpeta: `cd C:/Users/brown/Documents/software/universidad/CALIDAD DE SOFTWARE/CALIDAD DE SOFTWARE/helios-repo/frontend`
3. Correr: `pnpm vitest run`
4. La profesora vera en tiempo real como pasan los 55 tests con marquitas verdes

Opcion B - Si NO quieres usar terminal:

Muestra el archivo de test en GitHub. Por ejemplo:

- Abrir github.com/MarianC-programing/Helios

- Navegar a frontend/src/pages/Login.test.tsx
- La profesora vera las 11 funciones de test escritas con su descripcion

> *Frase: "Cada archivo .test.tsx contiene varios casos de prueba. Por ejemplo, este de Login tiene 11 tests que verifican el flujo de autenticacion."*

Como demostrar Locust

Locust no tiene plataforma cloud. Lo demuestras asi:

1. Abrir el archivo scripts/locustfile.py en GitHub
2. Mostrar el archivo de resultados (si lo tienes guardado) - captura con los 300ms P95

> *Frase: "El archivo locustfile.py define el escenario de prueba: 60% consultan horarios, 30% agendan, 10% revisan citas. Lo corrimos con 200 usuarios y obtuvimos P95 de 300ms."*

Plataformas con web (faciles)

GitHub: Pestana Commits muestra todos los cambios con fecha y autor. Branch sprint1/bootstrap.

Jira: Tablero Kanban con 3 columnas (Done, In Progress, To Do). 17 bugs + 5 historias.

Supabase: Menu izquierdo:

- Table Editor muestra las 6 tablas
- Authentication muestra usuarios registrados
- SQL Editor puede correr queries en vivo

Railway: Proyecto helios-citas, servicio helios-web en estado ACTIVE. Pestana Deployments muestra todos los despliegues con fecha.

Si NO puedes abrir alguna

Frase magica: "Profesora, todas las plataformas estan accesibles. Por restricciones de proyeccion/internet no puedo mostrarla en vivo ahora, pero le puedo enviar el link despues de la presentacion. Le parece?"

Recomendacion practica para el viernes

ANTES del demo:

1. Ten **abiertas en pestanas** del navegador:
2. github.com/MarianC-programing/Helios
3. helios-group.atlassian.net (Jira)
4. supabase.com/dashboard/...
5. railway.app/... proyecto helios
6. Ten un **terminal abierto** en la carpeta del proyecto, listo para correr pnpm vitest run.
7. Ten un **screenshot del reporte de Locust** guardado en el celular o pendrive.

SECCION 6: COMO EXPLICAR EL USO DE IA ASISTENTE

La verdad tecnica

Usamos asistencia de IA generativa para acelerar la escritura del codigo. Pero:

1. **El equipo decidio QUE probar** (login, registro, agendamiento, cedula panamena)
2. **El equipo definio los casos** (que pasa si la cedula es corta, si Supabase tarda, etc.)
3. **El equipo reviso CADA test** antes de aceptarlo
4. **El equipo los corrio y valido** que pasen
5. **El equipo los integro al repositorio**

La IA fue una herramienta de productividad, como un autocompletado avanzado. Igual que usar Stack Overflow, GitHub Copilot o ChatGPT para programar mas rapido.

Esto es 100% legitimo y aceptado en la industria del 2026. Empresas como Microsoft, Google, Amazon usan IA para asistir programacion profesional todos los dias.

Como lo explicas a la profesora - DOS versiones

Version A - HONESTA (recomendada)

> *"Usamos asistencia de IA generativa para acelerar la escritura del codigo boilerplate de los tests, igual que usamos GitHub Copilot o ChatGPT. Pero la **logica de negocio**, que probar, cuales son los casos criticos, y la **validacion** de cada test fue decision del equipo. Cada test fue revisado, ajustado y aprobado antes de integrarse. Es el flujo profesional moderno: el ingeniero define el QUE y el POR QUE, la IA ayuda con el COMO."*

Por que recomiendo esta: profesora del 2026 sabe perfectamente que se usa IA. Mentir es peor. Reconocer uso responsable demuestra madurez profesional.

Version B - NEUTRAL (si no quieres mencionar IA)

> *"Los tests los escribimos como equipo usando herramientas modernas de productividad. Cada test fue revisado y validado antes de integrarse al repositorio. Lo importante: los 55 tests pasan en cada deploy y cubren la logica critica del sistema."*

Si pregunta directamente: "Usaron ChatGPT o IA?"

NUNCA mientas. Di:

> *"Si profesora, usamos asistencia de IA como herramienta de productividad. Pero la responsabilidad tecnica, las decisiones de diseno, la revision de codigo y la validacion son del equipo. El estandar profesional actual es usar IA como copiloto, no como reemplazo."*

Eso es **honestidad + madurez tecnica**. Te ganas puntos, no los pierdes.

SECCION 7: PREGUNTAS ANTICIPADAS CON RESPUESTAS

Sobre conceptos

Que es ISTQB?

> *"Es el estandar internacional de calidad de software. Como ISO pero para pruebas."*

Que significa nivel 2 de TMMi?

> *"Es el modelo de madurez de pruebas del 1 al 5. Nivel 2 significa que tenemos proceso definido, plan documentado y code review obligatorio. No improvisamos. Estamos en camino a nivel 3."*

Que es TQM en una frase?

> *"Filosofia de mejora continua. Cada bug es una oportunidad para mejorar el proceso, no solo tapar el problema."*

Por que solo 61.4% de cobertura?

> *"Profesora, el estandar profesional es entre 60% y 80%. Nosotros enfocamos las pruebas en la logica critica: autenticacion, agendamiento y validaciones. Los archivos de estilos visuales no se prueban porque no tienen logica."*

Que es Latencia P95?

> *"Es una metrica de tiempo de respuesta. P95 significa que el 95% de las peticiones se atienden en menos de ese tiempo. Las nuestras en 300 milisegundos con 200 usuarios. La meta de la materia era 2 segundos."*

Como midieron eso?

> *"Con Locust, una herramienta de prueba de carga. Simula cientos de usuarios accediendo al sistema al mismo tiempo."*

Cuantos commits hizo cada miembro?

> *"Minimo uno firmado por cada integrante, 9 commits firmados en total, todos trazables en GitHub. Adicionalmente hay commits no firmados de tareas menores."*

Por que 17 bugs? No es mucho?

> *"Esa cifra es alta a proposito y la celebramos. Cada bug fue detectado por mi en uso real, no por usuarios desconocidos. Cada uno se arreglo en la misma jornada y se agrego un test de regresion. Si hubieramos escondido los bugs, la metrica seria 0 pero el sistema fallaria en produccion. Esto es honestidad profesional."*

Cual fue el bug mas complicado?

> *"El del flujo de registro de pacientes. Cuando alguien creaba su cuenta, a veces no se vinculaba correctamente con su perfil. Nos tomo varias iteraciones encontrar la causa real: un problema de tiempos entre la base de datos y la pantalla. Lo arreglamos antes del demo."*

Que es Plan Mode?

> *"Es nuestra regla obligatoria: antes de tocar codigo, disenamos un plan completo y lo aprobamos. Nadie improvisa. Esta en el archivo CLAUDE.md como regla del equipo."*

Que es la deuda tecnica que reconocen?

> *"Cosas que sabemos que faltan para produccion real: confirmacion por email, autenticacion de doble factor MFA, monitoreo con alertas. Las dejamos para el Sprint 2 porque este es un MVP academico."*

Que pasa si Supabase se cae?

> *"Tenemos timeouts de 15 segundos. Si Supabase no responde, sale un mensaje claro con boton de Reintentar. No se queda colgado eternamente."*

Como separan permisos de paciente y medico?

> *"Dos capas de seguridad. La primera en el frontend: cada rol ve pantallas distintas. La segunda en la base de datos con Row Level Security: aunque un atacante use DevTools del navegador para enviar consultas raras, la base de datos rechaza lo que no le corresponde."*

Que es una RPC atomica?

> *"Es una funcion en la base de datos que ejecuta varias operaciones como si fueran una sola. En nuestro caso, garantiza que dos pacientes NO puedan reservar el mismo horario al mismo tiempo, aunque hagan click al mismo milisegundo."*

Que es threshold automatico en CI?

> *"CI es el sistema que prueba el codigo cada vez que se sube. El threshold es un minimo de calidad. Si la cobertura baja del 60%, el sistema rechaza el deploy automaticamente. Es un guardia automatico."*

Esta listo para usuarios reales en produccion?

> *"Es un MVP academico. Para produccion comercial faltarian email de confirmacion, MFA y monitoreo profesional. Esta documentado en el roadmap del Sprint 2."*

Cuanto tardaron en hacerlo?

> *"Dos sprints de dos semanas. Sprint 0 fue setup, diseno y configuracion. Sprint 1 fue la implementacion completa del agendamiento end-to-end con dashboard de medicos."*

Sobre herramientas

Vitest tiene plataforma web?

> *"No, es CLI (linea de comandos). Puedo correrlo en mi laptop o mostrarle los archivos .test.tsx en GitHub."*

Locust tiene plataforma web?

> *"Tampoco, es script Python. El archivo locustfile.py esta en GitHub con los resultados documentados."*

SECCION 8: TIPS PARA NO PERDERTE EN LA PRESENTACION

Antes de hablar

- Respira hondo 3 veces
- Si te pones nervioso, agarra los 3 numeros clave: 55 tests, 300 ms, 17 bugs
- Tienes esta hoja a tu lado

Las 3 frases magicas si te trabas

1. "Esta documentado en el repositorio."
2. "Sigue las normas estandar de calidad de software."
3. "Es MVP. Hay roadmap para Sprint 2."

Si NO sabes algo

- "Esa es una excelente observacion. Lo investigamos para el Sprint 2."
- "Permitame consultar mis notas para darle el dato exacto."
- "Alejandro tiene el detalle tecnico, podrias complementar?"

Reglas basicas

- **Lento y claro** mas que rapido y nervioso
- Mira a la profesora
- Frases cortas
- Si necesitas leer, esta bien leer

Frase magica universal para herramientas

> *"La herramienta es estandar de la industria, pero la configuracion, los tests/scripts/migraciones que escribimos son codigo original del equipo. Esta todo en el repositorio."*

Esa frase te saca de cualquier apuro.

SECCION 9: RESUMEN ULTRA-RAPIDO

Cuando hables de la slide 7, sigue este orden mental:

1. "Aqui esta el registro completo" (mira la tabla izquierda)
2. **6 filas:** requerimientos, diseno, codigo, defectos, esfuerzo, productividad (3 segundos cada una)
3. "Usamos 6 herramientas" (senala el bloque derecho)
4. **4 herramientas con plataforma:** GitHub, Jira, Supabase, Railway
5. **2 herramientas CLI (sin plataforma web):** Vitest, Locust
6. "Tablero Kanban refleja el estado" (senala abajo derecha)
7. **3 columnas:** Done, In Progress, To Do

Los 3 numeros clave que NUNCA debes olvidar

Numero	Que es
55	Tests automatizados que pasan
300 ms	Latencia con 200 usuarios
17	Bugs detectados y resueltos

Si te bloqueas, regresa a esos 3 numeros y arranca de ahi.

FRASE DE CIERRE (slide 8)

> *"MVP funcional, metricas medidas, proceso documentado. Gracias. Preguntas?"*

Suerte el viernes, Jose. Vas preparado.